



DSA

Module 10

Linked Lists

Author
Srinivas Dande



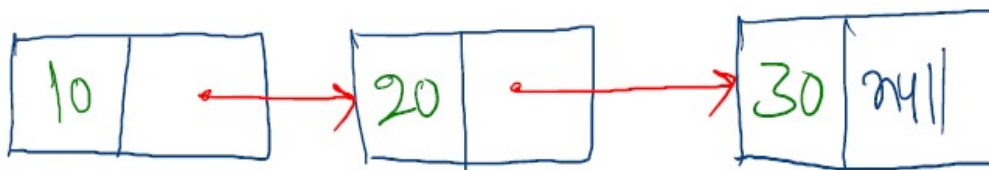
My **JAVA
LEARNING
CENTER**



11. Introduction to LinkedLists

- ♦ Linked List is Linear Data Structure.
- ♦ Linked List is Used to store collection of elements.
- ♦ LinkedLists elements can be accessed in sequential order only.
- ♦ LinkedLists elements will be stored with Node representation.

Ex:



- ♦ Linked list has the following properties.
 - ✓ Successive elements are connected by pointers
 - ✓ The last element points to NULL
 - ✓ Can grow or shrink in size during execution of a program
 - ✓ Does not waste memory space. It allocates memory as list grows but takes some extra memory for pointers.
- ♦ Types of Linked Lists
 - ✓ Singly Linked Lists
 - ✓ Doubly Linked Lists
 - ✓ Circular Linked Lists

11.1 Why LinkedLists

- ♦ There are many other data structures that do the same thing as linked lists.
- ♦ Before discussing linked lists it is important to understand the difference between linked lists and arrays.
- ♦ Both linked lists and arrays are used to store collections of data, and since both are used for the same purpose, we need to differentiate their usage.
- ♦ That means in which cases arrays are suitable and in which cases linked lists are suitable.



Advantages of Arrays:

- ♦ Arrays are Used to store collection of elements of Same type
- ♦ Arrays elements are stored in contiguous locations
- ♦ Arrays elements will be stored with index representaiton.
- ♦ Arrays allow Random access to elements.
- ♦ Search Operation is faster because we can apply Binary Search.
- ♦ Arrays are Cache Friendly.

DisAdvantages of Arrays:

- ♦ Arrays are fixed size.
It means Once you create the array, you can not increse the size.
- ♦ Momory waste when you allocate more space and store less elements.
- ♦ Momory shortage when you allocate less space and store more elements.
You need to resize the array when you want to store more elements.
Resizing is an expensive again.
- ♦ Insert Operation is expensive because it takes time to shift elements
- ♦ Delete Operation is expensive because it takes time to shift elements

Advantages of LinkedLists:

- ♦ LinkedLists are Used to store collection of elements
- ♦ LinkedLists elements will not be stored in contiguous locations
- ♦ LinkedLists elements will be stored with Node representaiton.
- ♦ LinkedLists allow Only sequential acess to elements, No Random access.
- ♦ Insert Operation is faster
- ♦ Delete Operation is faster
- ♦ LinkedLists are dynamic. i.e No Node Creation inadvance

DisAdvantages of LinkedLists:

- ♦ No Random access.
- ♦ Search Operation is expensive
we can not apply Binary Search on LinkedList because of Sequential Access.
- ♦ Momoeiy wastage for Node addresses.
- ♦ LinkedLists are Not Cache Friendly.



Comparison of Linked Lists with Arrays & Dynamic Arrays

<u>Parameter</u>	<u>Arrays</u>	<u>ArrayList</u>	<u>LinkedList</u>
Get Element	$O(1)$	$O(1)$	$O(n)$
Search Element	$O(\log n)$	$O(\log n)$	$O(n)$
Insert at Beginning	$O(n)$	$O(n)$	$O(1)$
Delete at Beginning	$O(n)$	$O(n)$	$O(1)$
Insert at End	$O(1)$	$O(1)$	$O(n)$
Delete at End	$O(1)$	$O(1)$	$O(n)$
Insert at Middle	$O(n)$	$O(n)$	$O(n)$
Delete at Middle	$O(n)$	$O(n)$	$O(n)$

Use-cases:

1. I am writing some programs which has to store very large number of elements. I may not get the contiguous memory because memory may be available as chunks. Which one to choose to store large number of elements.

Ans: LinkedList

2. Stacks and Queues can be implemented with Arrays and LinkedLists.
Ex: I want to Implement Round Robin Scheduling.

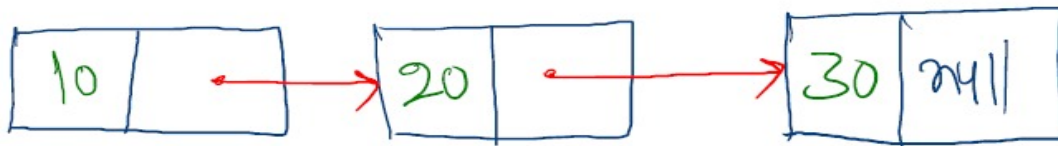
Ans: Use Queue with LinkedLists



11.2. Singly Linked List

- ♦ Singly Linked List consists of a number of nodes
- ♦ Each Node has will have 2 parts
 - a) Data
 - b) Address of Next Node
- ♦ Address of the last node in the list is NULL, which indicates the end of the list.

Ex:



- ♦ Following is a type declaration for a linked list:

```
public class Node {  
    int data;  
    Node next;  
  
    Node(int data) {  
        this.data = data;  
        this.next = null;  
    }  
}
```

- ♦ Main Linked Lists Operations
 - ✓ **Traverse:** Access the elements in the list
 - ✓ **Count:** returns the number of elements in the list
 - ✓ **Find** nth node from the end of the list
 - ✓ **Insert:** inserts an element into the list
 - ✓ **Delete:** removes the specified position element from the list



11.2.1. Traverse the LinkedList using Iterative Style

Lab1.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab1 {

    //Time Complexity - O(n)
    public static void displayList(Node headNode) {
        Node currNode = headNode;

        while(currNode!=null) {
            System.out.print(currNode.data+"\t");
            currNode = currNode.next;
        }
    }

    public static void main(String[] args) {

        Node head = new Node(10);
        head.next = new Node(20);
        head.next.next = new Node(30);
        head.next.next.next = new Node(40);
        head.next.next.next.next = new Node(50);

        displayList(head);
    }
}
```



11.2.2. Traverse the LinkedList using Recursive

Lab2.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab2 {

    // Time Complexity - O(n)
    public static void displayList(Node currentNode) {

        //Base Condition
        if(currentNode==null)
            return ;

        System.out.print(currentNode.data+"\t");
        displayList(currentNode.next); //Recursive Call
    }

    public static void main(String[] args) {

        Node head = new Node(10);
        head.next = new Node(20);
        head.next.next = new Node(30);
        head.next.next.next = new Node(40);
        head.next.next.next.next = new Node(50);

        displayList(head);
    }
}
```



11.2.3. Count the No. of Nodes in the LinkedList

Lab3.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab3 {
    // Time Complexity - O(n)
    static int length(Node headNode) {

        int length = 0;
        Node currentNode = headNode;
        while (currentNode != null) {
            length++;
            currentNode = currentNode.next;
        }
        return length;
    }

    public static void main(String[] args) {
        Node head = new Node(10);
        head.next = new Node(20);
        head.next.next = new Node(30);
        head.next.next.next = new Node(40);
        head.next.next.next.next = new Node(50);

        int len = length(head);
        System.out.println(len);

        Node myhead = null;
        int mylen = length(myhead);
        System.out.println(mylen);
    }
}
```



11.2.4. Insert the Node at First Position of LinkedList

Lab4.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab4 {
    // Time Complexity - O(1)
    public static Node insertFirst(Node headNode, int data) {

        Node temp = new Node(data);

        // 1. List is Empty
        if (headNode == null) {
            headNode = temp;
            return headNode;
        }

        // 2. List is Not Empty
        temp.next = headNode;
        headNode = temp;

        return headNode;
    }

    // Time Complexity - O(n)
    public static void displayList(Node headNode) {
        Node currNode = headNode;

        while (currNode != null) {
            System.out.println(currNode.data);
            currNode = currNode.next;
        }
    }
}
```



```
public static void main(String[] args) {

    Node myhead = null;
    myhead = insertFirst(myhead, 99);
    displayList(myhead);
    System.out.println("-----");

    myhead = insertFirst(myhead, 10);
    myhead = insertFirst(myhead, 20);
    myhead = insertFirst(myhead, 30);
    displayList(myhead);

}
}
```

11.2.5. Insert the Node at Last Position of LinkedList

Lab5.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab5 {

    // Time Complexity - O(n)
    public static Node insertLast(Node headNode, int data) {

        Node temp = new Node(data);

        // 1. List is Empty
        if (headNode == null) {
            headNode = temp;
            return headNode;
        }
    }
}
```



```
// 2. List is Not Empty
//Goto Last Node
Node currNode = headNode;
while(currNode.next!=null) {
    currNode = currNode.next;
}

currNode.next = temp;

return headNode;
}

// Time Complexity - O(n)
public static void displayList(Node headNode) {
    Node currNode = headNode;

    while (currNode != null) {
        System.out.println(currNode.data);
        currNode = currNode.next;
    }
}

public static void main(String[] args) {

    Node myhead = null;
    myhead = insertLast(myhead, 99);
    displayList(myhead);
    System.out.println("-----");

    myhead = insertLast(myhead, 10);
    myhead = insertLast(myhead, 20);
    myhead = insertLast(myhead, 30);
    displayList(myhead);
}
}
```



11.2.6. Insert the Node at given position of LinkedList

Lab6.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab6 {

    // Time Complexity - O(n)
    public static Node insert(Node headNode, int data, int position) {

        if (position <= 0) {
            return headNode;
        }

        Node temp = new Node(data);

        // 1. If position is 1
        if (position == 1) {
            temp.next = headNode;
            headNode = temp;

            return headNode;
        }

        // 1. If position >= 2
        // Goto Position Before Node
        Node currNode = headNode;
        int i = 1;
        while (i <= position - 2 && currNode != null) {
            currNode = currNode.next;
            i++;
        }
    }
}
```



```
        if (currNode == null) {
            return headNode;
        }

        temp.next = currNode.next;
        currNode.next = temp;

        return headNode;
    }

    // Time Complexity - O(n)
    public static void displayList(Node headNode) {
        Node currNode = headNode;

        while (currNode != null) {
            System.out.println(currNode.data);
            currNode = currNode.next;
        }
    }

    public static void main(String[] args) {

        Node head = new Node(10);
        head.next = new Node(20);
        head.next.next = new Node(30);
        head.next.next.next = new Node(40);
        head.next.next.next.next = new Node(50);
        displayList(head);
        System.out.println("-----");
        head = insert(head, 99, 4);
        displayList(head);
    }
}
```



11.2.7. Delete the First Node of LinkedList

Lab7.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab7 {

    // Time Complexity - O(1)
    public static Node deleteFirst(Node headNode) {

        // 1. List is Empty
        if (headNode == null) {
            return headNode;
        }

        // 2. List has only One Node
        if (headNode.next == null) {
            headNode = null;
            return headNode;
        }

        // 3. List has 2 or more Nodes
        Node temp = headNode;
        headNode = headNode.next;
        temp.next = null;

        return headNode;
    }

    public static Node insertLast(Node headNode, int data) {
        //Copy from Lab5
    }
}
```



```
public static void displayList(Node headNode) {  
    //Copy from Lab5  
}  
  
public static void main(String[] args) {  
  
    Node myhead = null;  
    myhead = insertLast(myhead, 99);  
    displayList(myhead);  
    System.out.println("-----");  
  
    myhead = deleteFirst(myhead);  
    displayList(myhead);  
    System.out.println("-----");  
  
    myhead = insertLast(myhead, 10);  
    myhead = insertLast(myhead, 20);  
    myhead = insertLast(myhead, 30);  
    displayList(myhead);  
    System.out.println("-----");  
  
    myhead = deleteFirst(myhead);  
    displayList(myhead);  
  
}  
}
```



11.2.8. Delete the Last Node of LinkedList

Lab8.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab8 {

    // Time Complexity - O(n)
    public static Node deleteLast(Node headNode) {

        // 1. List is Empty
        if (headNode == null) {
            return headNode;
        }

        // 2. List has only One Node
        if (headNode.next == null) {
            headNode = null;
            return headNode;
        }

        // 3. List has 2 or more Nodes
        Node currNode = headNode;
        while(currNode.next.next!=null) {
            currNode = currNode.next;
        }

        currNode.next = null;

        return headNode;
    }
}
```



```
public static Node insertLast(Node headNode, int data) {  
    //Copy from Lab5  
}  
  
public static void displayList(Node headNode) {  
    //Copy from Lab5  
}  
  
public static void main(String[] args) {  
  
    Node myhead = null;  
    myhead = insertLast(myhead, 99);  
    displayList(myhead);  
    System.out.println("-----");  
  
    myhead = deleteLast(myhead);  
    displayList(myhead);  
    System.out.println("-----");  
  
    myhead = insertLast(myhead, 10);  
    myhead = insertLast(myhead, 20);  
    myhead = insertLast(myhead, 30);  
    myhead = insertLast(myhead, 40);  
  
    displayList(myhead);  
    System.out.println("-----");  
  
    myhead = deleteLast(myhead);  
    displayList(myhead);  
  
}  
}
```



11.2.9. Delete the Node at given position of LinkedList

Lab9.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab9 {
    // Time Complexity - O(n)
    public static Node delete(Node headNode, int position) {

        // 1. position <= 0
        if (position <= 0)
            return headNode;

        // 2. If position is 1
        if (position == 1) {
            Node temp = headNode;
            headNode = headNode.next;
            temp.next = null;
            return headNode;
        }

        // 3. If position >= 2
        // Goto Position Before Node
        Node currNode = headNode;
        int i = 1;
        while (i <= position - 2 && currNode != null) {
            currNode = currNode.next;
            i++;
        }

        //4. Position > length
        if (currNode == null) {
            return headNode;
        }
    }
}
```



```
        Node temp = currNode.next;
        currNode.next = currNode.next.next;
        temp.next = null;

        return headNode;
    }
    public static Node insertLast(Node headNode, int data) {
        //Copy from Lab5
    }
    public static void displayList(Node headNode) {
        //Copy from Lab5
    }
    public static void main(String[] args) {

        Node myhead = null;
        myhead = insertLast(myhead, 99);
        displayList(myhead);

        myhead = delete(myhead,1);
        displayList(myhead);

        myhead = insertLast(myhead, 10);
        myhead = insertLast(myhead, 20);
        myhead = insertLast(myhead, 30);
        myhead = insertLast(myhead, 40);
        myhead = insertLast(myhead, 50);

        displayList(myhead);
        myhead = delete(myhead,4);
        displayList(myhead);
        myhead = delete(myhead,3);
        displayList(myhead);
    }
}
```



11.2.10. Search the Node in LinkedList

Lab10.java

```
package com.jlcindia.linkedlist;

public class Lab10 {
    // Time Complexity - O(n)
    public static int search(Node headNode, int element) {

        // 1. Empty List
        if (headNode == null)
            return -1;

        int position = 1;
        Node currNode = headNode;
        while (currNode != null) {
            if(currNode.data==element) {
                return position;
            }else {
                currNode = currNode.next;
                position++;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        Node head = new Node(10);
        head.next = new Node(20);
        head.next.next = new Node(30);
        head.next.next.next = new Node(40);
        head.next.next.next.next = new Node(50);

        int position = search(head, 50);
        System.out.println(position);
    }
}
```



11.3. Design MySinglyLinkedList

Lab11.java

```
package com.jlcindia.linkedlist;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class MySinglyLinkedList {

    Node headNode;
    int nodeCount;

    public MySinglyLinkedList() {
        this.headNode = null;
        this.nodeCount = 0;
    }

    public String toString() {
        if (this.headNode == null) {
            return "[]";
        }
        String str = "[";
        Node currNode = headNode;
        while (currNode != null) {
            str = str + currNode.data + ",";
            currNode = currNode.next;
        }
        str = str.substring(0, str.length() - 1);
        str = str + "]";
        return str;
    }

    public boolean isEmpty() {
        return nodeCount == 0;
    }
}
```



```
public int size() {
    return nodeCount;
}
public void insertFirst(int data) {

    Node temp = new Node(data);

    // 1. List is Empty
    if (this.headNode == null) {
        this.headNode = temp;
        nodeCount++;
        return;
    }

    // 2. List is Not Empty
    temp.next = this.headNode;
    this.headNode = temp;
    nodeCount++;
}
public void insertLast(int data) {
    Node temp = new Node(data);

    // 1. List is Empty
    if (this.headNode == null) {
        this.headNode = temp;
        nodeCount++;
        return;
    }
    // 2. List is Not Empty
    // Goto Last Node
    Node currNode = this.headNode;
    while (currNode.next != null) {
        currNode = currNode.next;
    }
    currNode.next = temp;
    nodeCount++;
}
```



```
public void insert(int position, int data) {

    // 1. If Position<=0
    if (position <= 0) {
        return;
    }

    Node temp = new Node(data);

    // 2. If position is 1
    if (position == 1) {
        insertFirst(data);
    }

    // 3. If position >= 2
    // Goto Position Before Node
    Node currNode = this.headNode;
    int i = 1;
    while (i <= position - 2 && currNode != null) {
        currNode = currNode.next;
        i++;
    }
    //4. If position is Out of Range
    if (currNode == null) {
        return ;
    }

    temp.next = currNode.next;
    currNode.next = temp;
    nodeCount++;
}
```



```
public void deleteFirst() {  
  
    // 1. List is Empty  
    if (this.headNode == null) {  
        return ;  
    }  
    // 2. List has only One Node  
    if (this.headNode.next == null) {  
        this.headNode = null;  
        nodeCount--;  
        return ;  
    }  
    // 3. List has 2 or more Nodes  
    Node temp = this.headNode;  
    this.headNode = this.headNode.next;  
    temp.next = null;  
    nodeCount--;  
}  
public void deleteLast() {  
    // 1. List is Empty  
    if (this.headNode == null) {  
        return ;  
    }  
    // 2. List has only One Node  
    if (this.headNode.next == null) {  
        this.headNode = null;  
        nodeCount--;  
        return ;  
    }  
    // 3. List has 2 or more Nodes  
    Node currNode = this.headNode;  
    while(currNode.next.next!=null) {  
        currNode = currNode.next;  
    }  
    currNode.next = null;  
    nodeCount--;  
}
```



```
public void delete( int position) {

    // 1. position <=0
    if (position <= 0) {
        return ;
    }

    // 2. If position is 1
    if (position == 1) {
        deleteFirst();
    }

    // 3. If position >= 2
    // Goto Position Before Node
    Node currNode = this.headNode;
    int i = 1;
    while (i <= position - 2 && currNode != null) {
        currNode = currNode.next;
        i++;
    }

    //4. Position > length
    if (currNode == null) {
        return ;
    }

    if(currNode.next!=null){
        Node temp = currNode.next;
        currNode.next = currNode.next.next;
        temp.next = null;
        nodeCount--;
    }
}
```



```
public int search(int element) {
    // 1. Empty List
    if (this.isEmpty())
        return -1;

    int position = 1;
    Node currNode = this.headNode;
    while (currNode != null) {
        if(currNode.data==element) {
            return position;
        }else {
            currNode = currNode.next;
            position++;
        }
    }
    return -1;
}

public boolean contains(int element) {

    // 1. Empty List
    if (this.isEmpty())
        return false;

    Node currNode = this.headNode;
    while (currNode != null) {
        if(currNode.data==element) {
            return true;
        }else {
            currNode = currNode.next;
        }
    }
    return false;
}
```



```
public void clear() {  
    Node currNode = this.headNode;  
    while(currNode!=null) {  
        Node temp = currNode.next;  
  
        currNode.next = null;  
        currNode = temp;  
    }  
    this.headNode=null;  
    this.nodeCount=0;  
}
```

} //End of Class

```
public class Lab11 {  
    public static void main(String[] args) {  
  
        MySinglyLinkedList mylist = new MySinglyLinkedList();  
  
        System.out.println(mylist);  
        System.out.println(mylist.isEmpty());  
        System.out.println(mylist.size());  
  
        mylist.insertFirst(20);  
        System.out.println(mylist);  
  
        mylist.insertFirst(10);  
        System.out.println(mylist);  
        System.out.println(mylist.isEmpty());  
        System.out.println(mylist.size());  
  
        mylist.insertLast(30);  
        mylist.insertLast(40);  
        mylist.insertLast(50);  
        System.out.println(mylist);  
        System.out.println(mylist.size());  
    }  
}
```



```
mylist.insert(3, 99);
mylist.insert(3, 88);

System.out.println(mylist);
System.out.println(mylist.size());

mylist.deleteFirst();
mylist.deleteLast();

System.out.println(mylist);
System.out.println(mylist.size());

mylist.delete(3);
mylist.delete(2);
mylist.delete(99);

System.out.println(mylist);
System.out.println(mylist.size());

System.out.println(mylist.search(30));
System.out.println(mylist.search(60));
System.out.println(mylist.contains(30));
System.out.println(mylist.contains(60));

System.out.println(mylist);
System.out.println(mylist.size());
mylist.clear();
System.out.println(mylist);
System.out.println(mylist.size());
    }
}
```



11.4. Design MyLinkedList - LeetCode - 707

Lab12.java

```
package com.jlcindia.linkedlists.leetcode;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Node {
    public int val;
    public Node next;

    public Node(int val) {
        this.val=val;
        this.next = null;
    }
}

class MyLinkedList {
    Node headNode;

    public MyLinkedList() {
        this.headNode = null;
    }

    public void addAtHead(int data) {
        Node temp = new Node(data);

        // 1. List is Empty
        if (this.headNode == null) {
            this.headNode = temp;
            return;
        }
        // 2. List is Not Empty
        temp.next = this.headNode;
        this.headNode = temp;
    }
}
```



```
public void addAtTail(int data) {
    Node temp = new Node(data);

    // 1. List is Empty
    if (this.headNode == null) {
        this.headNode = temp;
        return;
    }

    // 2. List is Not Empty
    // Goto Last Node
    Node currNode = this.headNode;
    while (currNode.next != null) {
        currNode = currNode.next;
    }
    currNode.next = temp;
}

public void addAtIndex(int index, int data) {

    int position = index+1;

    // 1. If Position<=0
    if (position <= 0) {
        return;
    }

    Node temp = new Node(data);

    // 2. If position is 1
    if (position == 1) {
        addAtHead(data);
        return;
    }
}
```



```
// 3. If position >= 2
// Goto Position Before Node
Node currNode = this.headNode;
int i = 1;
while (i <= position - 2 && currNode != null) {
    currNode = currNode.next;
    i++;
}
//4. If position is Out of Range
if (currNode == null) {
    return ;
}
temp.next = currNode.next;
currNode.next = temp;
}

public void deleteFirst() {
    // 1. List is Empty
    if (this.headNode == null) {
        return ;
    }

    // 2. List has only One Node
    if (this.headNode.next == null) {
        this.headNode = null;
        return ;
    }

    // 3. List has 2 or more Nodes
    Node temp = this.headNode;
    this.headNode = this.headNode.next;
    temp.next = null;
}
```



```
public void deleteAtIndex( int index) {

    int position = index+1;

    // 1. position <=0
    if (position <= 0) {
        return ;
    }

    // 2. If position is 1
    if (position == 1) {
        deleteFirst();
        return;
    }

    // 3. If position >= 2
    // Goto Position Before Node
    Node currNode = this.headNode;
    int i = 1;
    while (i <= position - 2 && currNode != null) {
        currNode = currNode.next;
        i++;
    }

    //4. Position > length
    if (currNode == null) {
        return ;
    }
    If(currNode.next!=null){
    Node temp = currNode.next;
    currNode.next = currNode.next.next;
    temp.next = null;
    }

}
```



```
public int get(int index) {

    int reqPosition = index+1;

    // 1. Empty List
    if (this.headNode==null)
        return -1;

    int currPosition = 1;
    Node currNode = this.headNode;
    while (currNode != null) {
        if(currPosition==reqPosition) {
            return currNode.val;
        }else {
            currNode = currNode.next;
            currPosition++;
        }
    }
    return -1;
}

} //End of Class

class Lab12 {
    public static void main(String[] args) {

        MyLinkedList mylist = new MyLinkedList();
        System.out.println(mylist);

        mylist.addAtHead(10);
        mylist.addAtTail(20);
        System.out.println(mylist);

        mylist.addAtIndex(0, 88);
        mylist.addAtIndex(1, 99);
        System.out.println(mylist);
    }
}
```



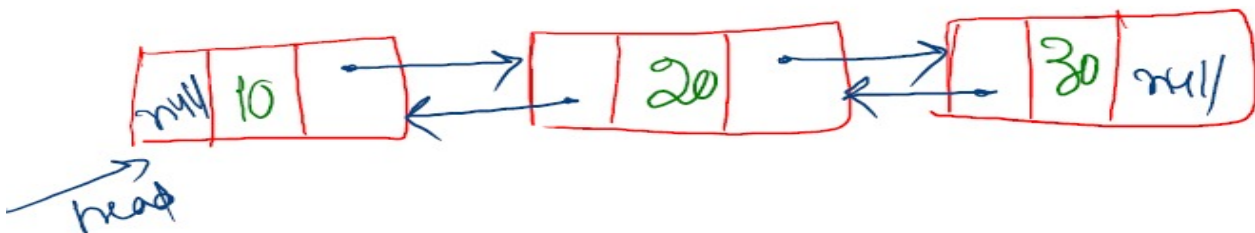
```
mylist.addAtTail(30);  
mylist.addAtTail(40);  
mylist.addAtTail(50);  
System.out.println(mylist);  
  
mylist.deleteAtIndex(0);  
System.out.println(mylist);  
  
mylist.deleteAtIndex(0);  
System.out.println(mylist);  
  
mylist.deleteAtIndex(3);  
System.out.println(mylist);  
  
System.out.println(mylist.get(0));  
System.out.println(mylist.get(1));  
System.out.println(mylist.get(2));  
  
    }  
}
```



11.5. Doubly Linked List

- ◆ Doubly Linked List consists of a number of nodes
- ◆ Each Node has will have 3 parts
 - a) Address of Previous Node
 - b) Data
 - c) Address of Next Node
- ◆ Next Address of the Last node in the list is NULL, which indicates that No Next Element.
- ◆ Previous Address of the First/Head node in the list is NULL, which indicates that No Previous element.

Ex:



- ◆ Following is a type declaration for a linked list:

```
public class Node {

    int data;
    Node next;
    Node prev;

    Node(int data) {
        this.data = data;
        this.next = null;
        this.prev=null;
    }
}
```



- ♦ Main Doubly Linked Lists Operations
 - ✓ Traverse: Access the elements in the List
 - ✓ Count: returns the number of elements in the List
 - ✓ Find nth node from the end of the List
 - ✓ Insert: inserts an element into the List
 - ✓ Delete: removes the specified position element from the List
 - ✓ Reverse the List

11.5.1. Traverse the Doubly LinkedList in forward order

Lab13.java

```
package com.jlcindia.linkedlist.doubly;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Node {

    int data;
    Node next;
    Node prev;

    Node(int data) {
        this.data = data;
        this.next = null;
        this.prev=null;
    }

}
```



```
public class Lab13 {  
    //Time Complexity - O(n)  
    public static void displayList(Node headNode) {  
        Node currNode = headNode;  
  
        while(currNode!=null) {  
            System.out.println(currNode.data);  
            currNode = currNode.next;  
        }  
    }  
    public static void main(String[] args) {  
  
        Node head = new Node(10);  
        Node node2= new Node(20);  
        Node node3= new Node(30);  
        Node node4 = new Node(40);  
  
        // 1st Node  
        head.prev = null;  
        head.next = node2;  
  
        // 2nd Node  
        node2.prev = head;  
        node2.next = node3;  
  
        // 3rd Node  
        node3.prev = node2;  
        node3.next = node4;  
  
        // 4th Node  
        node4.prev = node3;  
        node4.next = null;  
        displayList(head);  
    }  
}
```



11.5.2. Traverse the Doubly LinkedList in Reverse order

Lab14.java

```
package com.jlcindia.linkedlist.doubly;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab14 {

    // Time Complexity - O(2n)/O(n)
    public static void displayList(Node headNode) {

        Node currNode = headNode;

        // Move to Last Node
        while (currNode.next != null) {
            currNode = currNode.next;
        }

        while (currNode != null) {
            System.out.println(currNode.data);
            currNode = currNode.prev;
        }
    }

    public static void main(String[] args) {

        Node head = new Node(10);
        Node node2 = new Node(20);
        Node node3 = new Node(30);
        Node node4 = new Node(40);
```



```
// 1st Node
head.prev = null;
head.next = node2;

// 2nd Node
node2.prev = head;
node2.next = node3;

// 3rd Node
node3.prev = node2;
node3.next = node4;

// 4th Node
node4.prev = node3;
node4.next = null;

displayList(head);

    }
}
```



11.5.3. Insert the Node at beginning of D.L.L

Lab15.java

```
package com.jlcindia.linkedlist.doubly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab15 {

    public static Node insertFirst(Node headNode,int element) {

        Node temp = new Node(element);

        //1. List is Empty
        if(headNode==null) {
            headNode = temp; //Head Changed
            return headNode;
        }

        //2. List is Not Empty
        temp.next=headNode;
        headNode.prev=temp;
        headNode = temp; //Head Changed
        return headNode;

    }

    public static void displayList(Node headNode) {

        if(headNode==null) {
            System.out.println("List is Empty");
            return;
        }

        Node currNode = headNode;
```



```
        while(currNode!=null) {
            System.out.println(currNode.data);
            currNode = currNode.next;
        }
    }

    public static void main(String[] args) {

        Node head = null;
        head = insertFirst(head,10);
        head = insertFirst(head,20);
        head = insertFirst(head,30);
        displayList(head);
        System.out.println("-----");
        head = insertFirst(head,99);
        displayList(head);

    }
}
```



11.5.4. Insert the Node at end of D.L.L

Lab16.java

```
package com.jlcindia.linkedlist.doubly;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab16 {

    public static Node insertLast(Node headNode,int element) {

        Node temp = new Node(element);

        //1. List is Empty
        if(headNode==null) {
            headNode = temp; //Head Changed
            return headNode;
        }

        //2. List is Not Empty
        Node currNode = headNode;

        //Move to Last Node
        while(currNode.next!=null) {
            currNode = currNode.next;
        }

        currNode.next=temp;
        temp.prev=currNode;

        return headNode;
    }
}
```



```
public static void displayList(Node headNode) {

    if(headNode==null) {
        System.out.println("List is Empty");
        return;
    }

    Node currNode = headNode;

    while(currNode!=null) {
        System.out.println(currNode.data);
        currNode = currNode.next;
    }
}

public static void main(String[] args) {

    Node head = null;
    displayList(head);
    System.out.println("-----");
    head = insertLast(head,10);
    head = insertLast(head,20);
    head = insertLast(head,30);
    displayList(head);
    System.out.println("-----");
    head = insertLast(head,99);
    displayList(head);

}
```



11.5.5. Insert the Node at given position in D.L.L

Lab17.java

```
package com.jlccindia.linkedlist.doubly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab17 {
    public static Node insert(Node headNode,int element,int position) {
        if (position <= 0) {
            return headNode;
        }

        Node temp = new Node(element);

        // 1. If position is 1
        if (position == 1) {
            temp.next=headNode;
            headNode.prev=temp;
            headNode = temp; //Head Changed
            return headNode;
        }

        // 2. If position >= 2
        // Goto Position Before Node
        Node currNode = headNode;
        int i = 1;
        while (i <= position - 2 && currNode != null) {
            currNode = currNode.next;
            i++;
        }

        //3. Position is OOR
        if (currNode == null) {
            return headNode;
        }
    }
}
```



```
        temp.next = currNode.next;
        currNode.next.prev=temp;

        temp.prev=currNode;
        currNode.next = temp;

        return headNode;
    }

    public static Node insertLast(Node headNode,int element) {
        //Same as Lab16
    }

    public static void displayList(Node headNode) {
        //Same as Lab16
    }

    public static void main(String[] args) {

        Node head = null;
        head = insertLast(head,10);
        head = insertLast(head,20);
        head = insertLast(head,30);
        head = insertLast(head,40);
        displayList(head);
        System.out.println("-----");
        head = insert(head,99,2);
        displayList(head);

    }
}
```



9.3.6. Delete the First Node in D.L.L

Lab18.java

```
package com.jlcindia.linkedlist.doubly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab18 {

    static Node deleteFirst(Node headNode) {

        //1. If head is null
        if(headNode==null) {
            return null;
        }

        //2. If One Node is there
        if(headNode.next==null){
            return null;
        }

        //3.If Head is Not Null
        Node temp = headNode;

        headNode = headNode.next;
        headNode.prev=null;

        temp.next = null;

        return headNode;
    }

    static Node insertLast(Node headNode,int data) {
        // Copy from Lab15
    }
}
```



```
static String toString(Node headNode) {  
    // Copy from Lab15  
}  
  
public static void main(String[] args) {  
  
    Node head=null;  
    head = insertLast(head,10);  
    head = insertLast(head,20);  
    head = insertLast(head,30);  
    head = insertLast(head,40);  
    head = insertLast(head,50);  
  
    System.out.println(toString(head));  
    head = deleteFirst(head); //10  
    System.out.println(toString(head));  
    head = deleteFirst(head); //20  
    System.out.println(toString(head));  
  
}  
}
```



9.3.7. Delete the Last Node in D.L.L

Lab19.java

```
package com.jlcindia.linkedlist.doubly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab19 {

    static Node deleteLast(Node headNode) {

        //1. If head is null
        if(headNode==null) {
            return null;
        }

        //2. If One Node is there
        if(headNode.next==null){
            return null;
        }

        //3. Move to Last but one
        Node currentNode = headNode;
        while (currentNode.next.next != null) {
            currentNode = currentNode.next;
        }

        Node temp = currentNode.next;
        temp.prev=null;

        currentNode.next = null;
        return headNode;
    }
}
```



```
static Node insertLast(Node headNode,int data) {  
    // Copy from Lab15  
}  
  
static String toString(Node headNode) {  
    // Copy from Lab15  
}  
  
public static void main(String[] args) {  
  
    Node head=null;  
    head = insertLast(head,10);  
    head = insertLast(head,20);  
    head = insertLast(head,30);  
    head = insertLast(head,40);  
    head = insertLast(head,50);  
  
    System.out.println(toString(head));  
    head = deleteLast(head); //50  
    System.out.println(toString(head));  
    head = deleteLast(head); //40  
    System.out.println(toString(head));  
  
}  
}
```



9.3.8. Delete the Node at given position in D.L.L

Lab20.java

```
package com.jlcindia.linkedlist.doubly;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab20 {

    static Node delete(Node headNode, int position) {

        // 1.head is null
        if (headNode == null)
            return null;

        // 2.One Node
        if (headNode.next == null)
            return null;

        // 3.delete first node
        if (position == 1) {
            Node temp = headNode;

            headNode = headNode.next;
            headNode.prev = null;

            temp.next = null;
            return headNode;
        }
    }
}
```



```
// 4. Move to Position -1
Node currentCode = headNode;

for (int i = 1; i <= position - 2 && currentCode != null; i++) {
    currentCode = currentCode.next;
}

// 5.Position is out of Range
if (currentCode == null || currentCode.next==null) {
    return headNode;
}

Node temp = currentCode.next;

currentCode.next = currentCode.next.next;
if (currentCode.next != null) {
    currentCode.next.prev = currentCode;
}

temp.next = null;
temp.prev = null;

return headNode;
}

static Node insertLast(Node headNode,int data) {
    // Copy from Lab15
}

static String toString(Node headNode) {
    // Copy from Lab15
}
```



```
public static void main(String[] args) {  
  
    Node head = null;  
    head = insertLast(head, 10);  
    head = insertLast(head, 20);  
    head = insertLast(head, 30);  
    head = insertLast(head, 40);  
    head = insertLast(head, 50);  
  
    System.out.println(toString(head));  
    head = delete(head, 3); // 30  
    System.out.println(toString(head));  
    head = delete(head, 4); // 50  
    System.out.println(toString(head));  
    head = delete(head, 4); // No Deletion  
    System.out.println(toString(head));  
    head = delete(head, 3); // 40  
    System.out.println(toString(head));  
}  
}
```



9.3.9. Search the Node in D.L.L from beginnig

Lab21.java

```
package com.jlcindia.linkedlist.doubly;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab21{

    static int search(Node headNode, int element) {

        int position =1;

        Node currentNode = headNode;

        while(currentNode!=null) {
            if( currentNode.data == element) {
                return position;
            }else {
                position++;
                currentNode = currentNode.next;
            }
        }
        return -1;
    }

    static Node insertLast(Node headNode,int data) {
        // Copy from Lab15
    }

    static String toString(Node headNode) {
        // Copy from Lab15
    }
}
```



```
public static void main(String[] args) {  
  
    Node head = null;  
    head = insertLast(head, 10);  
    head = insertLast(head, 20);  
    head = insertLast(head, 30);  
    head = insertLast(head, 40);  
    head = insertLast(head, 50);  
  
    System.out.println(toString(head));  
    int x = search(head, 30); //3  
    System.out.println(x);  
    x = search(head, 50); //5  
    System.out.println(x);  
    x = search(head, 60); //-1  
    System.out.println(x);  
}  
}
```



9.3.10. Search the Node in D.L.L from ending

Lab22.java

```
package com.jlcindia.linkedlist.doubly;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab22{

    static int search(Node headNode, int element) {

        Node currentNode = headNode;
        int nodeCount = 1;

        //Move to Last Node
        while(currentNode.next != null) {
            currentNode = currentNode.next;
            nodeCount++;
        }

        while(currentNode!=null) {
            if( currentNode.data == element) {
                return nodeCount;
            }else {
                nodeCount--;
                currentNode = currentNode.prev;
            }
        }

        return -1;
    }
}
```



```
static Node insertLast(Node headNode,int data) {  
    // Copy from Lab15  
}
```

```
static String toString(Node headNode) {  
    // Copy from Lab15  
}
```

```
public static void main(String[] args) {  
  
    Node head = null;  
    head = insertLast(head, 10);  
    head = insertLast(head, 20);  
    head = insertLast(head, 30);  
    head = insertLast(head, 30);  
    head = insertLast(head, 30);  
    head = insertLast(head, 40);  
    head = insertLast(head, 50);  
  
    System.out.println(toString(head));  
    int x = search(head, 30); //3  
    System.out.println(x);  
    x = search(head, 50); //5  
    System.out.println(x);  
    x = search(head, 70); //-1  
    System.out.println(x);  
}
```

```
}
```



9.3.11. Reverse the D.L.L

Lab23.java

```
package com.jlcindia.linkedlist.doubly;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab23 {
    static Node reverse(Node headNode) {

        // 1. Empty List
        if (headNode == null) {
            return null;
        }
        // 2. One Node List
        if (headNode.next == null) {
            return headNode;
        }

        Node tempNode = null;

        Node currentNode = headNode;
        while(currentNode!=null) {
            //Swap
            tempNode = currentNode.prev;
            currentNode.prev= currentNode.next;
            currentNode.next = tempNode;

            currentNode = currentNode.prev;
        }

        headNode = tempNode.prev;
        return headNode;
    }
}
```



```
static Node insertLast(Node headNode,int data) {  
    // Copy from Lab15  
}  
  
static String toString(Node headNode) {  
    // Copy from Lab15  
}  
  
public static void main(String[] args) {  
  
    Node head = null;  
    head = insertLast(head, 10);  
    head = insertLast(head, 20);  
    head = insertLast(head, 30);  
    head = insertLast(head, 40);  
    head = insertLast(head, 50);  
  
    System.out.println(toString(head));  
    head = reverse(head);  
    System.out.println(toString(head));  
  
}  
}
```